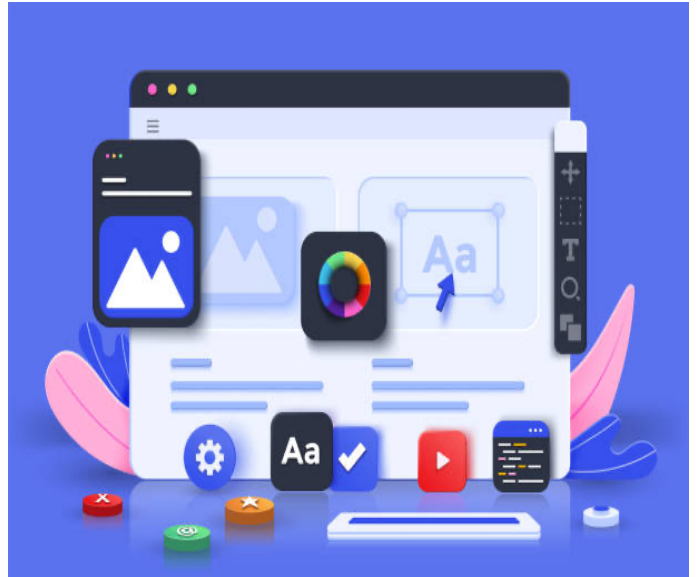


SD



Module Mijn eerste webgame

Instructie 01 – Voorbereidende activiteiten voor een project

- Introductie & Brainstormen
- Projectstructuur met mappen en bestanden, tools e.d.
- Basiskennis verwerven

Auteur: Johan Strootman

Afkorting: SMN

Versie: juli 2023

Inhoudsopgave

Inhoudsopgave	3
INLEIDING	5
Stappenplan	5
Waarom een stappenplan?	5
Welk stappenplan volgen wij?	5
BRAINSTORMEN.....	7
Inleiding	7
Wanneer wint een speler of is er een gelijkspel	8
Winnen	9
Wanneer is er gelijkspel?	9
ANALYSE VAN DE UI.....	10
Inleiding	10
Analyse van ons design (UI)	11
Verklaring	11
Speelveld.....	11
Kleur roze.....	11
Kleur lichtblauw	12
Kleur groen	12
ANALYSE SPELERSGEDRAG	13
Inleiding	13
Schets (Schema)	14
BASISKENNIS VERWERVEN.....	16
Inleiding	16

METHODE 1: Inline JavaScript.....	16
METHODE 2: Apart JavaScript bestand laden in de HEAD sectie van een pagina.....	17
Methode 2a	17
Methode 2b	17
METHODE 3: Aparte JavaScript bestand laden direct boven de sluittag van BODY.	18
Welke methode heeft dan de voorkeur?	18
Welke methode wordt dan het meest toegepast?	18
HOE VERWERKT DE BROWSER HTML?	19
Inleiding	19
Het DOM	22
Object	22
Voorbeeld	23
Verwerking van het voorbeeld door de browser	23
Visualisatie van het DOM na verwerking van het voorbeeld	24
WAT HEEFT JAVASCRIPT DAN MET HET DOM TE MAKEN?	25

INLEIDING

Stappenplan

Wij weten inmiddels dat we het spel Boter-Kaas-Eieren gaan maken met HTML5, CSS3 en JavaScript.

Om professioneel te werken aan dit project doorlopen we eerst een aantal voorbereidende stappen alvorens we feitelijk aan het coderen slaan.

Om gestructureerd aan het werk te gaan en daarmee voor onszelf het overzicht niet verliezen gaan we een stappenplan volgen. Tijdens de instructiemomenten gaan we de stappen niet exact volgen wegens gebrek aan tijd, maar zodra je aan je eigen project voor deze module gaat werken is het noodzakelijk dat je de stappen volgt.

Waarom een stappenplan?

In iedere organisatie wordt gebruik gemaakt van een 'stappenplan' tijdens het ontwikkelen van applicaties. Dit zorgt er voor dat ieder teamlid weet wat hem/haar te wachten staat, in welke fase een project zit, welke afspraken er gemaakt zijn en meer. Zonder een stappenplan kan een team niet samenwerken aan een project. Ook wanneer je in je eentje aan een project werkt is het noodzakelijk. Al snel hebben developers de neiging om gelijk te gaan programmeren. Wanneer ze dat doen komen ze er ALTIJD achter dat ze het overzicht kwijtgeraakt zijn en niet meer weten hoe nu verder. Dit resulteert vaak ook in code wat rommelig, inconsequent is en niet meer te onderhouden. Wat ook vaak gebeurd is dat men tijdens het coderen nog steeds allerlei zijsprongen moet maken om dingen te regelen of klaar te zetten alvorens verder te kunnen met coderen. Dit leidt enorm af en breekt de workflow waarin je zit tijdens het coderen. Het is dus altijd noodzakelijk, maar zeker ook 'Best Practice', om met een stappenplan te werken om dit soort situaties te voorkomen.

Welk stappenplan volgen wij?

Een basic stappenplan welke je kan vinden op It's Learning op de startpagina van de module in het blok EXTRA MATERIAAL.

LET OP!!!!

- *De resultaten van iedere stap in dit stappenplan moeten door jezelf verwerkt worden in een projectdocument. Dit projectdocument maakt deel uit van het assessment.*
- *De instructiemomenten zijn niet bedoeld om samen met jou als student een volledig geprogrammeerd en werkende applicatie af te leveren. Ze zijn bedoeld om je inzichten te geven in de werkwijze, te leren hoe je HTML5, CSS3 en JavaScript in een groter geheel kunt samenbrengen om een werkende applicatie te krijgen en om je te leren zelf na te denken over oplossingen voor problemen waar je tegenaan loopt.*

BRAINSTORMEN

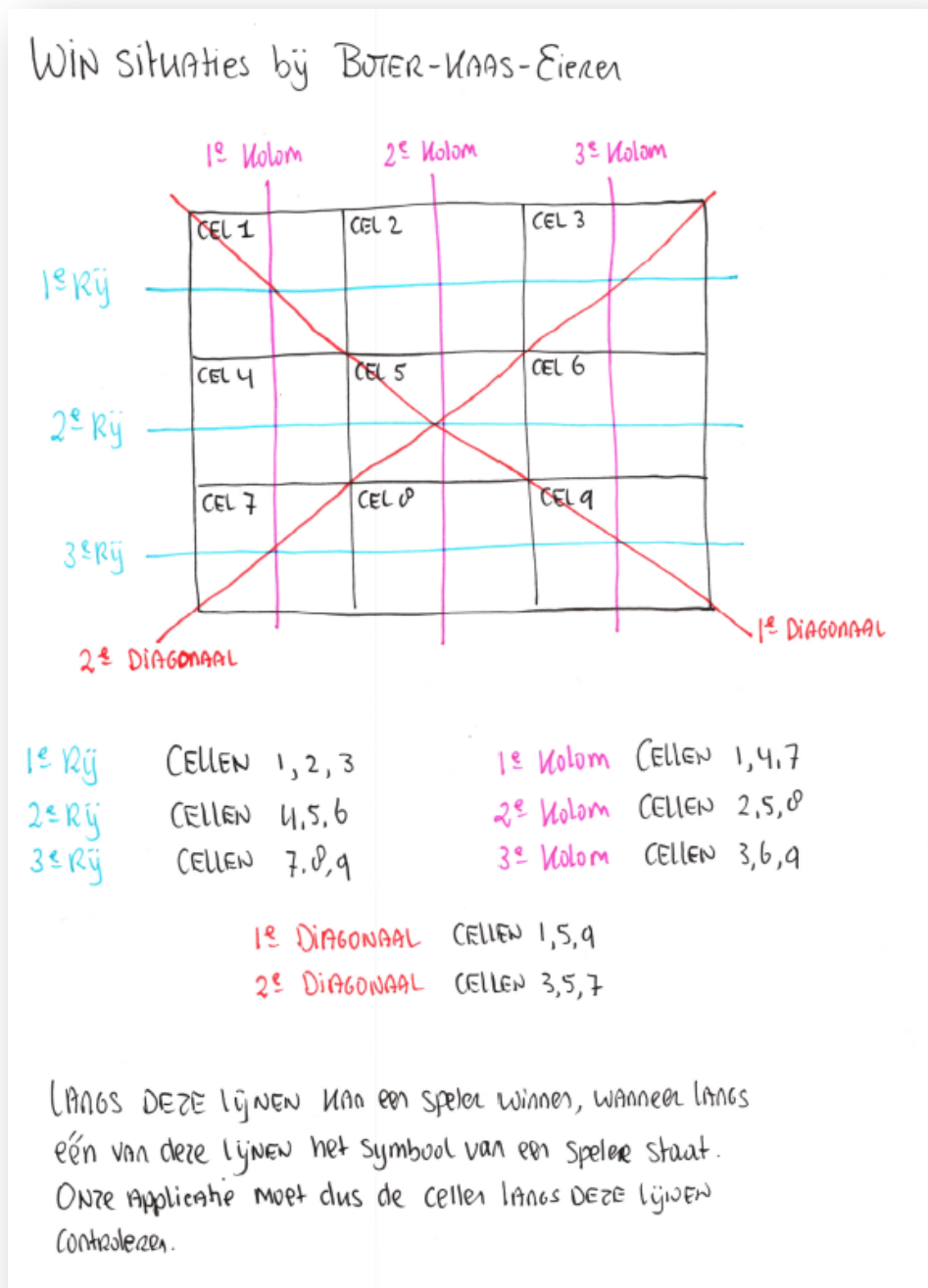
Inleiding

We kijken nu naar een aantal aspecten van het stappenplan, namelijk het product zelf en de gebruikers acties.

Dit houdt in dat we eerst een goed beeld moeten krijgen van wat we nu eigenlijk moeten gaan maken. En als dat beeld duidelijk is gaan we dit verder in detail uitspitten. Ook gaan we kijken welke invloed gedrag van gebruikers op onze applicatie kunnen hebben.

Aangezien we het spel Boter-Kaas-Eieren maken gaan we ons verdiepen in het spel zelf, wanneer wint een speler, wanneer is er sprake van een gelijkspel, hoe zit het met punten toekennen aan een winnaar en zo.

Wanneer wint een speler of is er een gelijkspel



Figuur 1 - Schets van de winsituaties

Allereerst schetsen we dit:

Winnen

We zien nu dat een speler kan winnen als de speler langs één van de lijnen z'n symbool heeft staan.

Dit levert ons dus 8 lijnen in totaal op.

Dit betekent automatisch ook dat onze applicatie langs die 8 lijnen moet controleren of een speler gewonnen heeft.

Wanneer is er gelijkspel?

Een gelijkspel is er wanneer het speelveld helemaal gevuld is met symbolen maar geen van beide spelers langs één van de 8 lijnen heeft gewonnen.

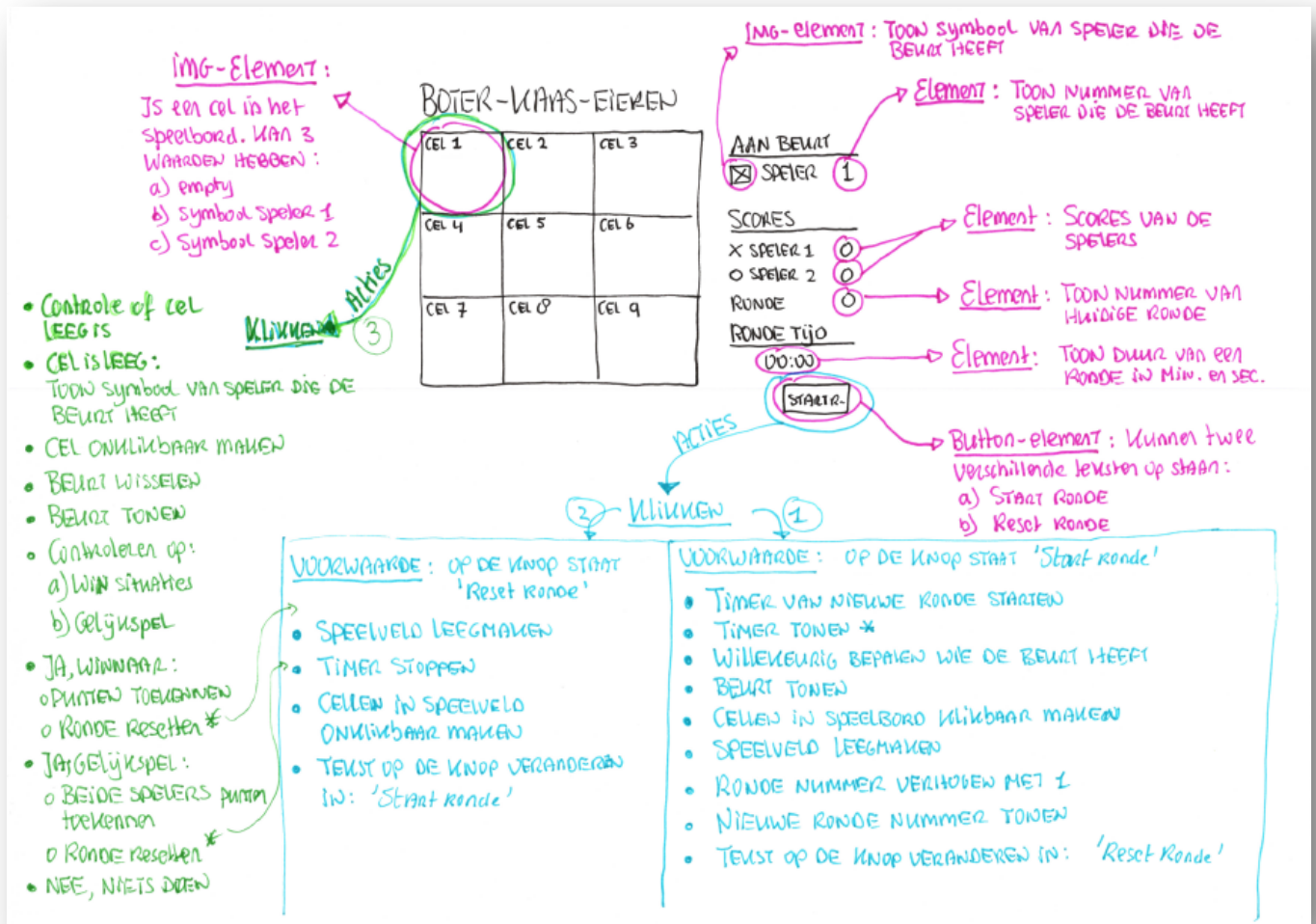
ANALYSE VAN DE UI

Inleiding

In één van de voorbereidende stappen hebben we een ontwerp gemaakt (wireframe en grafisch ontwerp). We weten dus nu hoe onze applicatie er uit gaat zien.

Toch moeten we de UI nog analyseren. We kunnen namelijk uit de UI afleiden welke elementen een rol gaan spelen door b.v. de content van die elementen dynamisch te maken en welke elementen een rol gaan spelen tijdens het spelen van het spel. Wat we in het laatste geval inzichtelijk willen hebben is wat er dan moet gebeuren.

Analyse van ons design (UI)



Figuur 2 - UI met annotaties na analyse

Verklaring

Speelveld

We hebben ontdekt en besloten dat het speelveld uit 9 IMG-tags gaat bestaan en dat we 3 afbeeldingen gaan gebruiken voor de weergave van een lege cel, een cel bezet door speler 1 en een cel bezet door speler 2.

Kleur roze

De elementen die gedurende het spelen van het spel aan wijzigingen vanuit onze applicatie onderhevig zijn. Dus vanuit onze code (JavaScript) moeten we bij deze elementen kunnen om hun content aan te kunnen passen.

Kleur lichtblauw

Een beschrijving van de acties die moeten plaatsvinden wanneer een speler op de knop klikt en daarbij zijn de te nemen acties afhankelijk van welke tekst er op de knop staat. We onderscheiden hier twee acties:

- **ACTIE 1**

De dingen die moeten gebeuren wanneer op de knop geklikt wordt terwijl de tekst '*Start ronde*' op de knop staat.

- **ACTIE 2**

De dingen die moeten gebeuren wanneer op de knop geklikt wordt terwijl de tekst '*Reset ronde*' op de knop staat.

Kleur groen

Hier beschrijven we de dingen die moeten gebeuren wanneer een speler op een cel in het speelveld klikt.

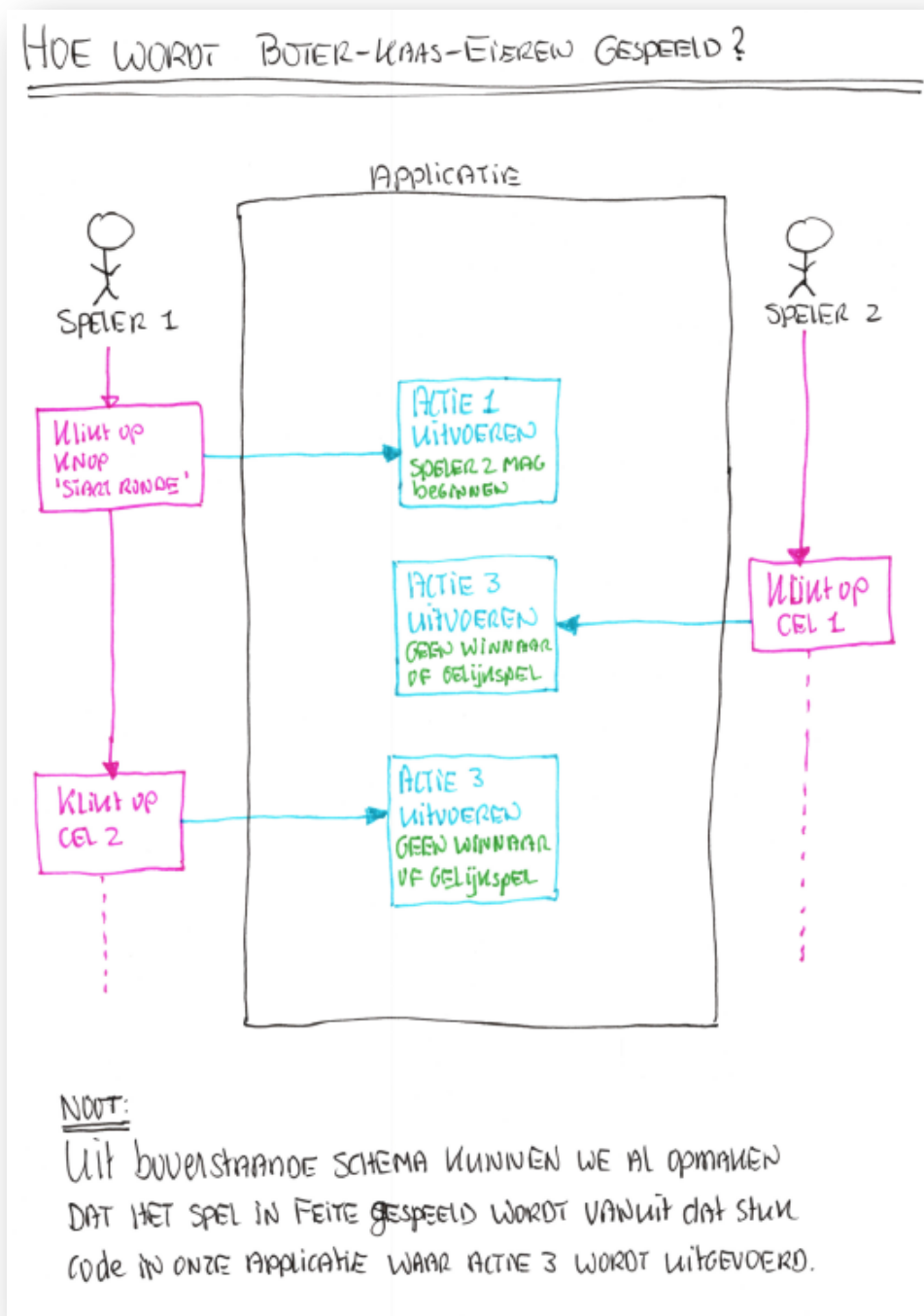
Dit noemen we **ACTIE 3**.

ANALYSE SPELERSGEDRAG

Inleiding

Om inzicht te krijgen in hoe het spel gespeeld wordt schetsen we dit. We verplaatsen ons daarbij in de rol van een speler.

We krijgen dan eveneens inzicht in wat en wanneer onze applicaties iets moet doen.



Figuur 3 - Voorbeeld schets spelers activiteiten en gedrag

Als we iedere stap volgen die een speler (of spelers) doet om het spel te spelen in kaart brengen zien we nog duidelijker hoe we onze applicatie moeten gaan schrijven.

Zo zien we nu al dat een bepaald stuk code in ons programma het in feite mogelijk maakt om het spel te spelen. Het klikken op de button leidt niet tot het spelen van het spel, dit is alleen nodig om een ronde te starten b.v. Het spel wordt pas echt gespeeld met de code die bij ACTIE 3 hoort. Dit zal waarschijnlijk een functie worden in onze code (althans voor zover we nu al kunnen zien).

BASISKENNIS VERWERVEN

Inleiding

Er zijn meerdere methoden om je JavaScript code te koppelen aan een HTML-pagina.

De methoden zijn:

METHODE 1: Inline JavaScript

Op deze manier plaatsen we de JavaScript code rechtstreeks tussen de tags `<script>` en `</script>` in de HTML-pagina.

Dit heeft echter niet de voorkeur. Voor volledige applicaties geschreven in JavaScript doen we dit nooit.

We plaatsen de code altijd in aparte JavaScript bestanden.

METHODE 2: Apart JavaScript bestand laden in de HEAD sectie van een pagina

Met het plaatsen van de SCRIPT-tag in de **HEAD** sectie van een pagina vertellen we de Browser dat het genoemde JavaScript bestand ook nog moet worden geladen en worden uitgevoerd.

Ook hier zijn er echter weer twee manieren om dat te doen:

Methode 2a

```
<script type="text/javascript" src="js/index.js"></script>
```

Hierboven zie je als voorbeeld hoe het JavaScript bestand **index.js** in de map **js** gekoppeld wordt aan een HTML-pagina.

Deze manier kan echter problemen opleveren.

Een probleem ontstaat wanneer in de JavaScript code een element van de pagina wordt binnengehaald en de code daar gelijk iets mee moet doen.

Waarom is dit een probleem?

De Browser gaat van boven naar beneden door de HTML-code heen, komt deze SCRIPT-tag tegen en gaat, voordat de Browser verder gaat met het verwerken van de rest van de HTML-code, eerst dit JavaScript bestand van de server laden en daarna ook gelijk uitvoeren. Dit betekent dat de JavaScript code dan kan refereren naar een HTML-element op de pagina die op dat moment nog niet eens door de Browser verwerkt is. Dus kan de JavaScript code dit element niet vinden.

Methode 2b

```
<script type="text/javascript" src="js/index.js" defer></script>
```

Door het attribuut **defer** toe te voegen aan de SCRIPT-tag vertellen we de Browser dat deze eerst verder moet met het verwerken van de HTML-code en pas daarna de JavaScript code moet gaan uitvoeren. Dit voorkomt de problemen die we hierboven beschreven hebben.

METHODE 3: Aparte JavaScript bestand laden direct boven de sluittag van BODY.

We kunnen de SCRIPT-tag ook direct boven de sluittag van BODY plaatsen (</body>)

```
<script type="text/javascript" src="js/index.js"></script>  
</body>
```

Hierbij hoeven de attribuut defer niet te gebruiken. We weten nu zeker namelijk dat de Browser eerst alle HTML-code en CSS-code heeft verwerkt voordat de JavaScript code wordt geladen en uitgevoerd.

Welke methode heeft dan de voorkeur?

De methode 2b en methode 3 hebben de voorkeur. Welke van deze twee je gebruikt maakt in principe niet uit, het betreft een persoonlijke voorkeur.

Welke methode wordt dan het meest toegepast?

Methode 3 wordt het meest toegepast. We kunnen dit zien als 'Best Practice'.

HOE VERWERKT DE BROWSER HTML?

Inleiding

Wanneer je de URL van een website intikt in de adresbalk van de browser gaat de browser, door gebruik te maken van het **HTTP-protocol** (*een afsprakenlijst over hoe computers met elkaar moeten communiceren via het internet*), een **HTTP-request** versturen naar de webserver waar de website op staat. In dit request staat dan dat de Browser om de eerste webpagina vraagt van de website.

Het gaat hier dus om een simpele HTTP-vraag: *Geef mij a.u.b. de eerste pagina van de website*

Een dergelijke vraag wordt altijd gedaan met een **HTTP GET-request**.

Wanneer de Browser de eerste pagina van de website ontvangt van de webserver is dit altijd een bestand met HTML-code, want HTML-code bepaald hoe een pagina ingedeeld is.

```

1  <!DOCTYPE html>
2  <html lang="nl">
3
4      <head>
5          <meta charset="UTF-8">
6          <meta http-equiv="X-UA-Compatible" content="IE=edge">
7          <meta name="viewport" content="width=device-width, initial-scale=1.0">
8
9          <title>DOM Voorbeeld</title>
10
11         <link rel="stylesheet" href="css/dom-voorbeeld.css">
12     </head>
13
14     <body>
15         <div class="container">
16             <header>
17                 
18                 <ul>
19                     <li>
20                         <a href="dom-voorbeeld.html">
21                             Home
22                         </a>
23                     </li>
24                     <li>
25                         <a href="#">
26                             About
27                         </a>
28                     </li>
29                     <li>
30                         <a href="#">
31                             Contact
32                         </a>
33                     </li>
34                 </ul>
35             </header>
36
37             <main>
38                 <section>
39                     <h1>
40                         DOM Voorbeeld
41                     </h1>
42                     <p>
43                         Banner
44                     </p>
45                 </section>
46
47                 <section>
48                     <h1>
49                         Content deel van de website
50                     </h1>
51                     <p>
52                         Lorem ipsum, dolor sit amet consectetur adipisicing elit.
53                         Quibusdam id consectetur sed harum delectus deserunt voluptatibus
54                         itaque ullam totam in voluptates placeat ex, veritatis doloremque
55                         culpa reiciendis, nisi officiis. Soluta.
56                     </p>
57                 </section>
58             </main>
59         </div>
60
61         <footer>
62             <p>
63                 Dit is de footer
64             </p>
65         </footer>
66     </body>
67 </html>

```

Figuur 4 - Voorbeeld HTML-code voor de uitleg

De Browser verwerkt deze HTML-code **regel voor regel** en slaat dit in het **interne geheugen** van de computer op in de vorm van een **boomstructuur**. Hierbij is ieder element in de HTML-code een **object** (*voorwerp*) in de boomstructuur met **eigenschappen** en **functionaliteit**.

Over de manier waarop de Browser een dergelijke pagina verwerkt en wat dat met JavaScript te maken heeft leer je tijdens dit instructiemoment.

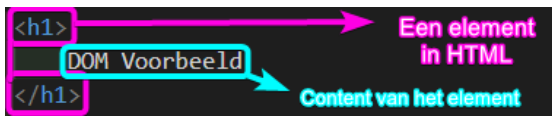
Het DOM

Het DOM, oftewel Document Object Model, beschrijft de structuur van een webpagina zoals de browser deze verwerkt en opslaat in het interne geheugen van een computer.

Object

Ieder element (b.v. een TAG in de HTML-code) wordt opgeslagen in de DOM als een object, oftewel een voorwerp, met eigenschappen en (niet noodzakelijk) functionaliteit.

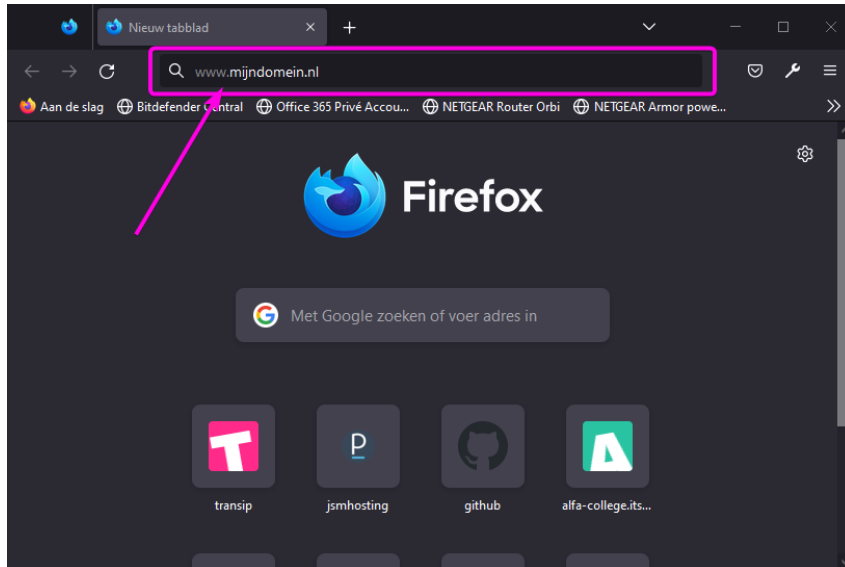
Bijvoorbeeld:



Het element is de H1-tag. Deze tag heeft eigenschappen, zoals kleur, lettergrootte en allerlei CSS gekoppelde eigenschappen. Deze tag heeft als CHILD content in de vorm van platte tekst.

Voorbeeld

Stel je tikt de volgende URL in de adresbalk van de browser in:



Figuur 5 - URL intikken in de adresbalk

Als alles goed gaat en de webserver reageert zoals verwacht dan wordt de eerste webpagina van deze website teruggestuurd naar de browser.

Verwerking van het voorbeeld door de browser

De browser gaat nu regel voor regel van boven naar beneden door de HTML-code en verwerkt dit in de DOM.

Op de 11e regel van de code komt de browser het volgende tegen:

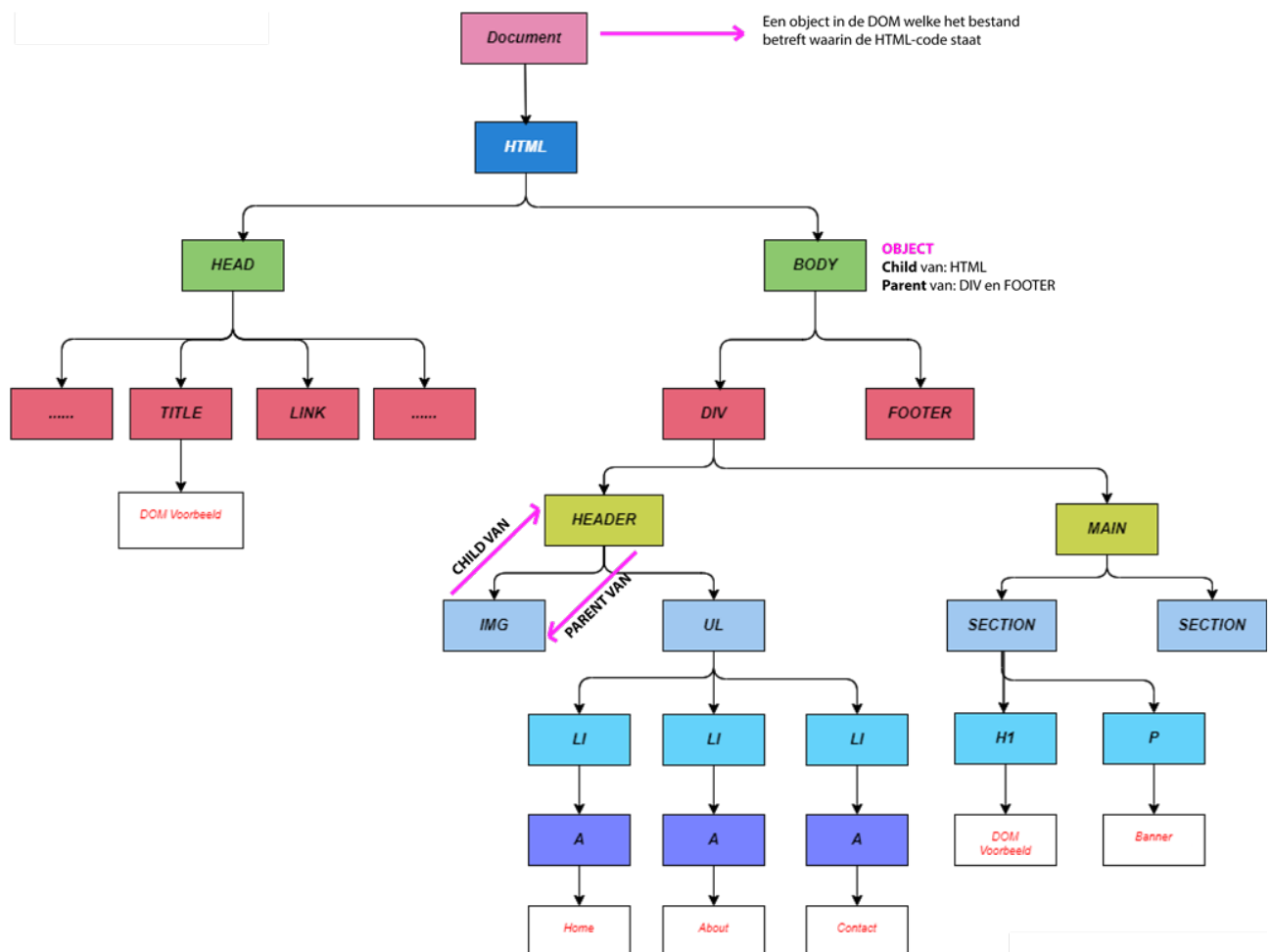
```
<link rel="stylesheet" href="css/dom-voorbeeld.css">
```

Voordat de browser verder gaat met verwerken vanaf de 12e regel verstuurd de browser weer een **HTTP GET-request** naar de webserver met het verzoek om het bestand **dom-voorbeeld.css** te mogen ontvangen.

Wanneer deze ontvangen is wordt de inhoud (CSS-code) van dit bestand eerst verwerkt, pas daarna gaat de browser verder vanaf de 12e regel HTML-code met verwerken van de HTML.

Visualisatie van het DOM na verwerking van het voorbeeld

Om de verwerking van de HTML (en CSS) code te visualiseren kunnen we het DOM tekenen in een boomstructuur.



Figuur 6 - Boomstructuur van de DOM

In dit voorbeeld (schema hierboven) ontbreekt natuurlijk het een en ander. We hebben namelijk in dit schema, wat slechts als voorbeeld dient, niet alle elementen in de HTML-code verwerkt. De browser doet dit uiteraard wel.

WAT HEEFT JAVASCRIPT DAN MET HET DOM TE MAKEN?

JavaScript code kan niet standaard bij de elementen op de pagina. Wanneer we iets willen doen met een element op de pagina moeten we dit element eerst '*binnenhalen*'. Daarvoor kent JavaScript een aantal opdrachten. Volg de link hieronder voor meer informatie.

[LINK: W3Schools - Binnenhalen van elementen](#)

Het '*binnenhalen*' van een element betekent niet dat we vanuit JavaScript rechtstreeks bij de HTML-code kunnen, maar betekent dat we elementen als objecten vanuit het DOM '*binnenhalen*'.

Dit betekent ook dat, zodra we dus een element als een object '*binnenhalen*', we gelijk ook alle eigenschappen en functies (die door de browser aan dit element in de DOM gekoppeld zijn) beschikbaar hebben.